```python
# ==============================================
# Wavelet + Circular Test + TE + AME Pipeline
# ==============================================

# ==== IMPORTS ====
import numpy as np
import pandas as pd
import pywt
from scipy.ndimage import gaussian_filter, uniform_filter

# ==== GLOBAL CONFIG (bạn có thể chỉnh) ====
RANDOM_STATE = 42
dt           = 1.0
wavelet_name = "cmor1.5-1.0"
min_scale, max_scale, num_scales = 2, 128, 64

# TE/AME & Permutation
LAG     = 1
N_BINS  = 6
N_PERM  = 500

# WTC significance + thực dụng
alpha_wtc = 0.05
coh_thr   = 0.70
min_run   = 6

# Nhãn (để ghi kết quả) — dùng đúng tên cột của bạn
x_label = "EPU_MoM"
y_label = "INF_MoM"

rng = np.random.default_rng(RANDOM_STATE)


df = pd.read_csv("")
# Chuẩn hóa thời gian & chọn đúng hai biến nghiên cứu
df = df.copy()
df["Time"] = pd.to_datetime(df["Time"], errors="coerce")
df =
df.dropna(subset=["Time"]).sort_values("Time").set_index("Time")
df = df[["EPU_MoM", "INF_MoM"]].apply(pd.to_numeric,
errors="coerce").dropna()

# Gán cho pipeline
t = df.index                            # DatetimeIndex
x = df["EPU_MoM"].to_numpy()            # nguồn
y = df["INF_MoM"].to_numpy()            # đích
assert len(t) == len(x) == len(y) and len(x) > 30, "Độ dài chuỗi
không hợp lệ."
```

```
# ==== WTC CORE (NEW)  ====
def _smooth2d(A, sigma_t=3, size_s=3):
    B = gaussian_filter(A, sigma=(0, sigma_t))      # (scale, time)
    B = uniform_filter(B, size=(size_s, 1))
    return B

def compute_wtc_arrays(x_vals, y_vals, dt=1.0,
wavelet='cmor1.5-1.0',
                       min_scale=2, max_scale=128, num_scales=64,
                       smooth_t=3, smooth_s=3):
    x0 = x_vals - np.nanmean(x_vals)
    y0 = y_vals - np.nanmean(y_vals)

    scales = np.linspace(min_scale, max_scale, num_scales)
    Wx, freqs = pywt.cwt(x0, scales, wavelet, dt)
    Wy, _     = pywt.cwt(y0, scales, wavelet, dt)
    period    = 1.0 / (freqs + 1e-12)

    Wxy = Wx * np.conj(Wy)
    Px  = np.abs(Wx)**2
    Py  = np.abs(Wy)**2

    Sx  = _smooth2d(Px / scales[:, None], sigma_t=smooth_t,
size_s=smooth_s)
    Sy  = _smooth2d(Py / scales[:, None], sigma_t=smooth_t,
size_s=smooth_s)
    Sxy = _smooth2d(Wxy / scales[:, None], sigma_t=smooth_t,
size_s=smooth_s)

    coh   = (np.abs(Sxy)**2) / (Sx * Sy + 1e-12)
    coh   = np.clip(coh, 0, 1)
    phase = np.angle(Wxy)  # pha từ XWT
    return coh, phase, period

def wtc_pvals_circular(x_series, y_series, n_perm=500, seed=42,
**wtc_kw):
    rng_local = np.random.default_rng(seed)

    # canh thẳng hàng một lần
    xy = pd.concat([x_series.rename('x'), y_series.rename('y')],
axis=1).dropna()
    times = xy.index
    x_vals = xy['x'].values
    y_vals = xy['y'].values
```

```
    coh_obs, phase, period = compute_wtc_arrays(x_vals, y_vals,
**wtc_kw)
    ge_counts = np.zeros_like(coh_obs, dtype=np.int32)
    T = len(y_vals)

    for _ in range(n_perm):
        shift = rng_local.integers(0, T)
        y_perm = np.roll(y_vals, shift)
        coh_perm, _, _ = compute_wtc_arrays(y_perm, x_vals,
**wtc_kw)  # coherence đối xứng
        ge_counts += (coh_perm >= coh_obs)

    pvals = (ge_counts + 1) / (n_perm + 1)  # add-one smoothing
    return coh_obs, phase, period, times, pvals


# ==== TE / AME tools  ====
def quantile_digitize(z, n_bins):
    z = np.asarray(z, float)
    mask = np.isfinite(z); zv = z[mask]
    if zv.size < n_bins + 5:
        mn, mx = np.nanmin(zv), np.nanmax(zv)
        edges = np.linspace(mn, mx + 1e-12, n_bins + 1)
    else:
        q = np.linspace(0, 1, n_bins + 1)
        edges = np.quantile(zv, q)
        edges = np.unique(edges)
        if edges.size - 1 < n_bins:
            mn, mx = np.nanmin(zv), np.nanmax(zv)
            edges = np.linspace(mn, mx + 1e-12, n_bins + 1)
    bins = np.full_like(z, -1, dtype=int)
    bins[mask] = np.clip(np.digitize(zv, edges[1:-1], right=False),
0, n_bins - 1)
    return bins

def transfer_entropy_discrete(x_s, y_s, lag=1, n_bins=5,
n_perm=1000, rng=None):
    y_t1 = y_s[lag:]; y_t = y_s[:-lag]; x_t = x_s[:-lag]
    bx = quantile_digitize(x_t, n_bins)
    by = quantile_digitize(y_t, n_bins)
    by1 = quantile_digitize(y_t1, n_bins)
    m = (bx >= 0) & (by >= 0) & (by1 >= 0)
    bx, by, by1 = bx[m], by[m], by1[m]
    if bx.size < 30:
        return 0.0, 1.0

    K = n_bins
    C = np.zeros((K, K, K), float)  # [y1,y,x]
```

```
    np.add.at(C, (by1, by, bx), 1.0)
    P = C / C.sum()
    Py1y = P.sum(2); Py = Py1y.sum(0); Pyx = P.sum(0)
    Py1_yx = P / (Pyx[None, :, :] + 1e-12)
    Py1_y  = Py1y / (Py[None, :] + 1e-12)
    TE_obs = float(np.nansum(P * np.log((Py1_yx + 1e-12) /
(Py1_y[:, :, None] + 1e-12))))

    greater = 0; n_valid = bx.size
    rng_local = rng if rng is not None else
np.random.default_rng(0)
    for _ in range(n_perm):
        k = int(rng_local.integers(1, n_valid))
        bx_p = np.roll(bx, k)
        Cp = np.zeros_like(C); np.add.at(Cp, (by1, by, bx_p), 1.0)
        Pp = Cp / Cp.sum(); Py1y_p = Pp.sum(2); Py_p =
Py1y_p.sum(0); Pyx_p = Pp.sum(0)
        Py1_yx_p = Pp / (Pyx_p[None, :, :] + 1e-12)
        Py1_y_p  = Py1y_p / (Py_p[None, :] + 1e-12)
        TE_p = np.nansum(Pp * np.log((Py1_yx_p + 1e-12) /
(Py1_y_p[:, :, None] + 1e-12)))
        if TE_p >= TE_obs:
            greater += 1
    p_val = (greater + 1) / (n_perm + 1)
    return TE_obs, float(p_val)

def ame_bootstrap(x_s, y_s, lag=1, n_boot=1000, rng=rng):
    y1 = y_s[lag:]; y0 = y_s[:-lag]; xx = x_s[:-lag]
    m = np.isfinite(y1) & np.isfinite(y0) & np.isfinite(xx)
    y1, y0, xx = y1[m], y0[m], xx[m]
    if y1.size < 30:
        return 0.0, 0.5, (np.nan, np.nan), 1.0
    X = np.column_stack([np.ones_like(xx), y0, xx])
    beta, *_ = np.linalg.lstsq(X, y1, rcond=None); ame =
float(beta[-1])
    ame_pos_share = float(np.mean((xx * ame) > 0))
    n = y1.size; B = min(n_boot, 1000)
    rng_local = rng if rng is not None else
np.random.default_rng(0)
    bs = np.empty(B)
    for b in range(B):
        idx = rng_local.integers(0, n, n)
        betab, *_ = np.linalg.lstsq(X[idx], y1[idx], rcond=None)
        bs[b] = betab[-1]
    lo, hi = np.percentile(bs, [2.5, 97.5])
    p_boot = 2 * min(np.mean(bs <= 0), np.mean(bs >= 0))
    return ame, ame_pos_share, (float(lo), float(hi)),
float(p_boot)
```

```
def _windows(mask, min_run=6):
    out = []
    i, n = 0, mask.size
    while i < n:
        if mask[i]:
            j = i
            while j + 1 < n and mask[j+1]:
                j += 1
            if j - i + 1 >= min_run:
                out.append((i, j))
            i = j + 1
        else:
            i += 1
    return out

def _fmt_windows(times_index, wins):
    if (wins is None) or (len(wins) == 0):
        return ""
    segs =
[f"{pd.to_datetime(times_index[a]).strftime('%Y-%m')}~{pd.to_datet
ime(times_index[b]).strftime('%Y-%m')}"
            for a, b in wins]
    return "; ".join(segs)


# ====================================
# ==== WTC + Circular p-values RUN ====
# ====================================
# Convert to Series (đảm bảo cùng index thời gian)
t_index = pd.to_datetime(t)
x_ser   = pd.Series(x.astype(float), index=t_index)
y_ser   = pd.Series(y.astype(float), index=t_index)

coh, phase, periods, times_wtc, pvals_wtc = wtc_pvals_circular(
    x_ser, y_ser,
    n_perm=N_PERM,
    seed=RANDOM_STATE,
    dt=dt, wavelet=wavelet_name,
    min_scale=min_scale, max_scale=max_scale,
num_scales=num_scales,
    smooth_t=3, smooth_s=3
)

# ==============================
# ==== Chuẩn bị Wx, Wy cho TE/AME (dùng real part theo từng scale
như bản gốc) ====
# ==============================
```

```
scales = np.linspace(min_scale, max_scale, num_scales)
x0 = x - np.nanmean(x)
y0 = y - np.nanmean(y)
Wx, _ = pywt.cwt(x0, scales, wavelet_name, dt)
Wy, _ = pywt.cwt(y0, scales, wavelet_name, dt)

# ========================
# ==== Tổng hợp kết quả ===
# ========================
rows = []
scale_range_str = f"{min_scale}-{max_scale}"

for j, (per, s) in enumerate(zip(periods, scales), start=1):
    # series tại scale j (real part) để TE/AME
    x_s = np.real(Wx[j-1, :])
    y_s = np.real(Wy[j-1, :])

    # coherence & phase tại scale j
    coh_j   = coh[j-1, :]
    phase_j = phase[j-1, :]
    sig_j   = (pvals_wtc[j-1, :] < alpha_wtc)  # ô có ý nghĩa theo
circular

    # [NEW] lấy p-value trung bình và nhỏ nhất của circular test ở
scale j
    wtc_p_mean = float(np.nanmean(pvals_wtc[j-1, :]))
    wtc_p_min  = float(np.nanmin(pvals_wtc[j-1, :]))

    coh_mean = float(np.nanmean(coh_j))
    coh_max  = float(np.nanmax(coh_j))

    # Cửa sổ theo chiều pha + ý nghĩa + ngưỡng thực dụng
    mask_xy = (np.isfinite(coh_j)) & sig_j & (coh_j >= coh_thr) &
(phase_j > 0)   # EPU -> INF
    mask_yx = (np.isfinite(coh_j)) & sig_j & (coh_j >= coh_thr) &
(phase_j < 0)   # INF -> EPU
    win_xy = _windows(mask_xy, min_run=min_run)
    win_yx = _windows(mask_yx, min_run=min_run)

    # ----- EPU_MoM -> INF_MoM -----
    TE_xy, p_xy = transfer_entropy_discrete(x_s, y_s, lag=LAG,
n_bins=N_BINS, n_perm=N_PERM, rng=rng)
    ame_xy, share_xy, (lo_xy, hi_xy), p_ame_xy = ame_bootstrap(x_s,
y_s, lag=LAG, n_boot=N_PERM, rng=rng)
    rows.append({
        "Period": float(per), "N_BINS": N_BINS, "LAG": LAG,
"Wavelet": wavelet_name,
```

```
        "Scale_Range": scale_range_str, "Scale_Level": j,
"Direction": f"{x_label}->{y_label}",
        "TE": float(TE_xy), "p_value": float(p_xy),
        "AME_mean": float(ame_xy), "AME_pos_share":
float(share_xy),
        "AME_CI95": (float(lo_xy), float(hi_xy)), "AME_p":
float(p_ame_xy),
        "coh_mean": coh_mean, "coh_max": coh_max,
        "WTC_p_mean": wtc_p_mean, "WTC_p_min": wtc_p_min,   # [NEW]
2 cột p-value circular
        "Time_Windows": _fmt_windows(times_wtc, win_xy),
        "x_label": x_label, "y_label": y_label
    })

    # ----- INF_MoM -> EPU_MoM -----
    TE_yx, p_yx = transfer_entropy_discrete(y_s, x_s, lag=LAG,
n_bins=N_BINS, n_perm=N_PERM, rng=rng)
    ame_yx, share_yx, (lo_yx, hi_yx), p_ame_yx = ame_bootstrap(y_s,
x_s, lag=LAG, n_boot=N_PERM, rng=rng)
    rows.append({
        "Time_Windows": _fmt_windows(times_wtc, win_yx),
        "N_BINS": N_BINS, "LAG": LAG,
        "Scale_Range": scale_range_str, "Scale_Level": j,
"Direction": f"{y_label}->{x_label}",
        "TE": float(TE_yx), "p_value": float(p_yx),
        "AME_mean": float(ame_yx), "AME_pos_share":
float(share_yx),
        "AME_CI95": (float(lo_yx), float(hi_yx)), "AME_p":
float(p_ame_yx),
        "coh_mean": coh_mean, "coh_max": coh_max,
        "WTC_p_mean": wtc_p_mean, "WTC_p_min": wtc_p_min,   # [NEW]
giữ nguyên cho hướng ngược lại
        "x_label": y_label, "y_label": x_label
    })


summary_df = pd.DataFrame(rows)

# Sắp xếp cột (giữ nguyên format bạn yêu cầu)
cols =
["Time_Windows","N_BINS","LAG","Scale_Range","Scale_Level","Direct
ion",

"TE","p_value","AME_mean","AME_pos_share","AME_CI95","AME_p",

"coh_mean","coh_max","WTC_p_mean","WTC_p_min","x_label","y_label"]
summary_df = summary_df[cols]
```

```
summary_df.to_csv("Wavelet_Circular_TE_AME_results.csv",
index=False)
print("✅ Đã lưu kết quả vào file:
Wavelet_Circular_TE_AME_results.csv")

# Done: summary_df chứa kết quả đã có circular test cho WTC +
TE/AME + Time_Windows
```